

Topic 7: Software Testing

Software Testing

- Testing is defined as the execution of a program to find its faults
- While more time is typically spent on testing than in any other phase of SDLC, there is considerable confusion about its purpose
- Many software professionals, for example, believe that tests are run to show that the program works rather than to learn about its faults

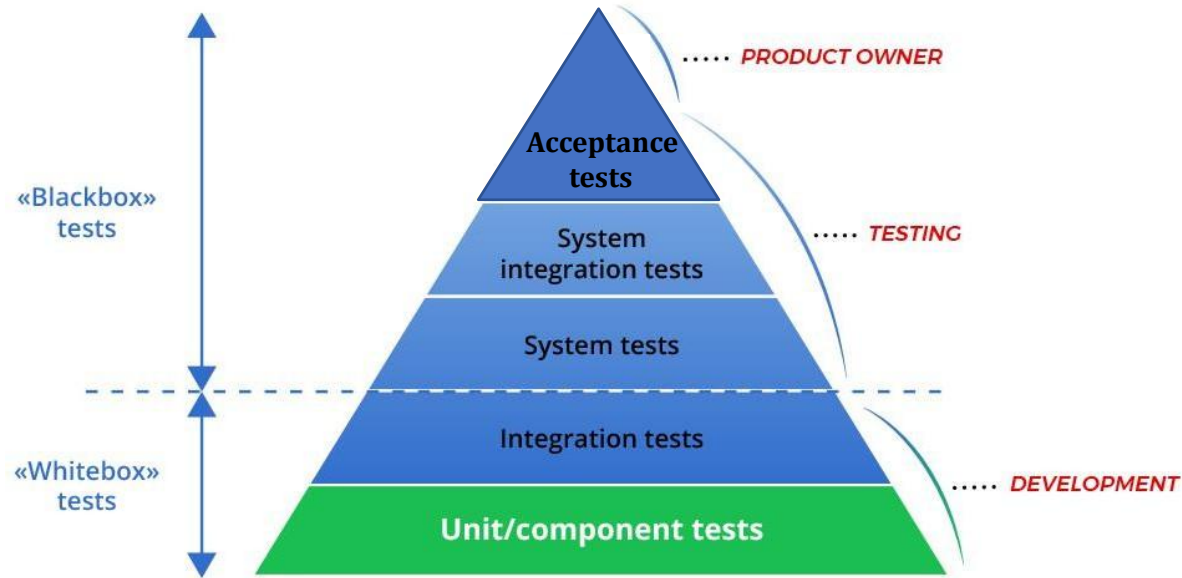
Software Testing Principles

- To remove as many defects as possible before test since the quality improvement potential of testing is limited
- Exhaustive testing is practically impossible
 - Example: 10-letter string $\rightarrow 26^{10}$ combinations
 - Testing 1 input per microsecond = ~ 4.5 million years needed!
- Reality
 - We can only test a small, smart subset of conditions that will reveal the characteristics of the program
- Therefore
 - Focus on early defect removal (reviews, verification)
 - Testing is for detection, not perfection

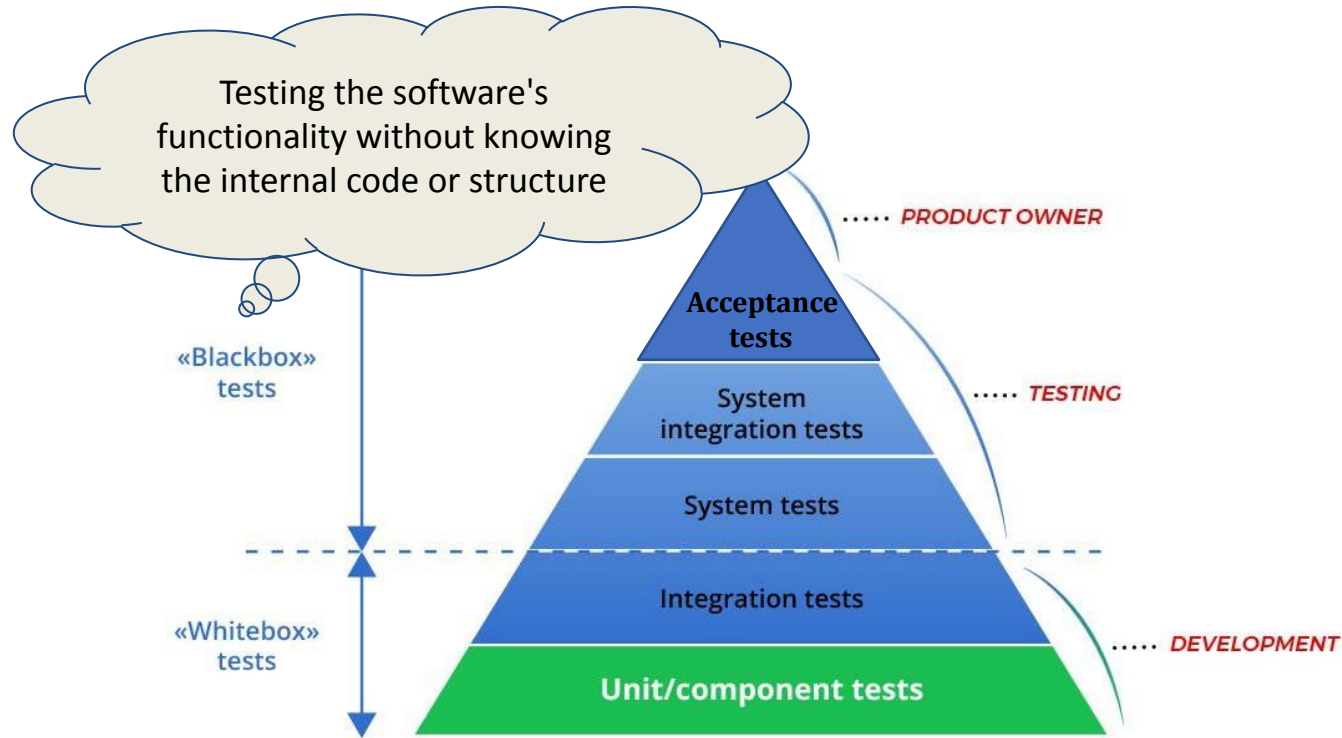
Types of Software Tests

- Unit Testing
- Integration Testing
- Function Testing
- Regression Testing
- System Testing
- Acceptance and Installation Tests

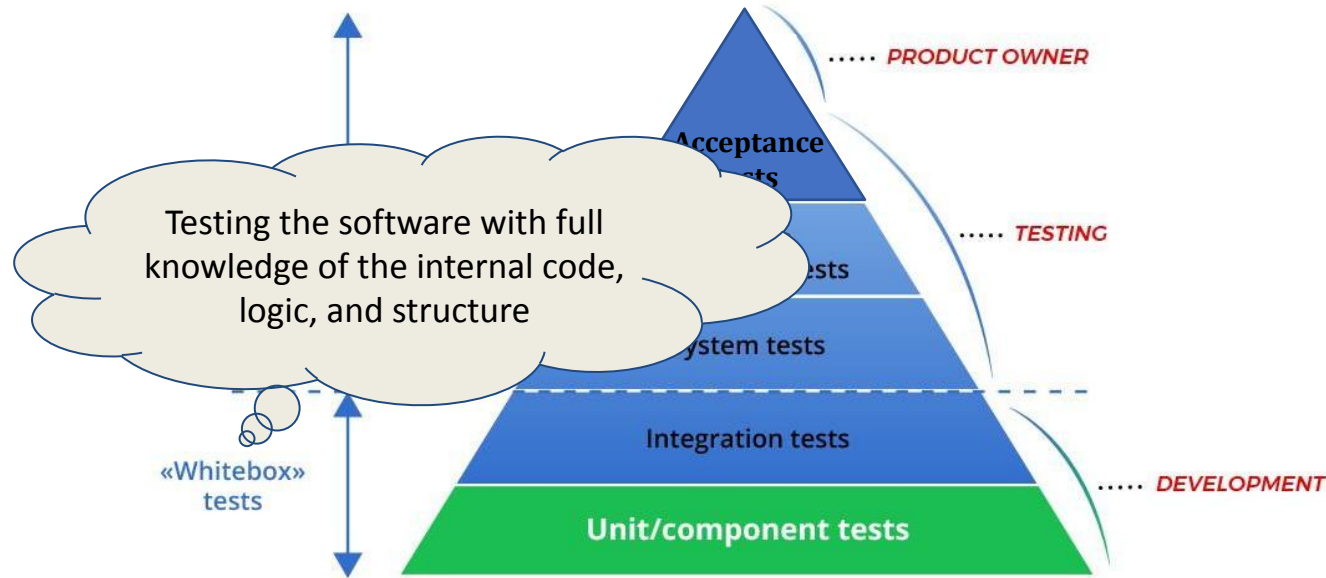
Types of Software Testing



Types of Software Testing



Types of Software Testing



Topic 7.1: White Box Testing

White Box Testing

- White-box testing, sometimes called glass-box testing, is a test-case design philosophy that uses the control structure described as part of component-level design to derive test cases.
- Different techniques:
 - Basis path testing
 - Control structure testing

Path Testing

- Path testing is based on control structure of the program for which flow graph is prepared.
- Path testing requires complete knowledge of the program's structure.
- Path testing is closer to the developer and used by him to test his module.
- The effectiveness of path testing gets reduced with the increase in size of software under test.
- Choose enough paths in a program such that maximum logic coverage is achieved.

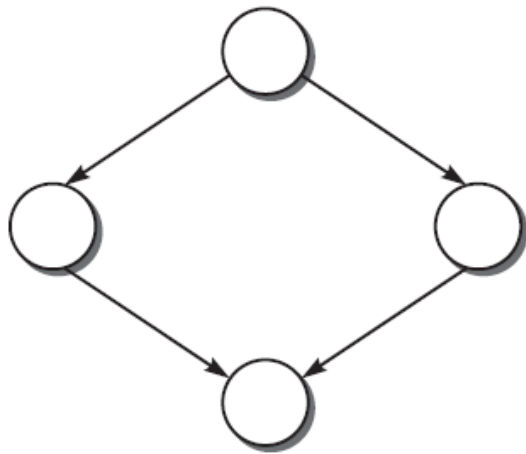
Control Flow Graph

- The control flow graph is a graphical representation of control structure of a program.
- Some Term:
 - **Node:** It represents one or more procedural statements. The nodes are denoted by a circle. These are numbered or labeled.
 - **Edges or links:** They represent the flow of control in a program. This is denoted by an arrow on the edge. An edge must terminate at a node.
 - **Decision node:** A node with more than one arrow leaving it is called a decision node.
 - **Junction node:** A node with more than one arrow entering it is called a junction.
 - **Regions:** Areas bounded by edges and nodes are called regions. When counting the regions, the area outside the graph is also considered a region.

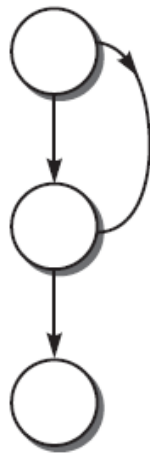
Flow Graph Notations



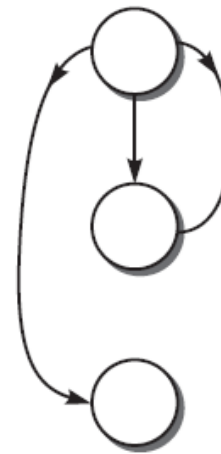
(a) Sequence



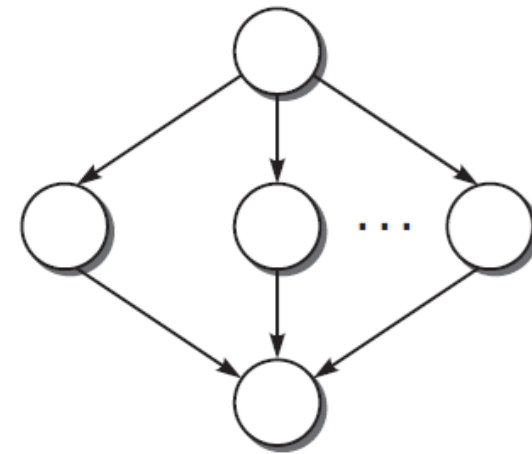
(b) If-Then-Else



(c) Do-While



(d) While-Do
/For Loop



(e) Switch-Case

PATH TESTING TERMINOLOGY

- Path
 - A path through a program is a sequence of instructions or statements that starts at an entry, junction, or decision and ends at another, or possibly the same, junction, decision, or exit.
- Segment
 - Paths consist of segments. The smallest segment is a link, that is, a single process that lies between two nodes (e.g., junction-process-junction, junction process-decision, decision-process-junction, decision-process-decision).
- Length of a Path
 - The length of a path is measured by the number of links in it and not by the number of instructions or statements executed along the path. An alternative way to measure the length of a path is by the number of nodes traversed.
- Independent Path
 - An independent path is any path through the graph that introduces at least one new set of processing statements or new conditions.

Cyclomatic Complexity (CC)

- Cyclomatic complexity is a software metric used to measure the complexity of a program.
- The idea is to measure the complexity by considering the number of paths in the control graph of the program.
- But even for simple programs, if they contain at least one cycle, the number of paths is infinite. Therefore, we will consider only independent paths.
- Cyclomatic complexity number can be derived through any of the following three formulae:

$$V(G) = e - n + 2p$$

where e is number of edges, n is the number of nodes in the graph, and p is number of components in the whole graph.

DD Graph (Decision-to-Decision Graph)

- For a DD graph, the following actions must be done:
 - Put the line numbers on the execution statements of the program. Start numbering the statements after declaring the variables, if no variables have been initialized. Otherwise, start from the statement where a variable has been initialized.
 - Put the sequential statements in one node. For example, statements 1, 2, and 3 have been put inside one node.
 - Put the edges between the nodes according to their flow of execution.
 - Put alphabetical numbering on each node like A, B, etc.

Example 1

- Draw the DD graph for the program.
- Calculate the cyclomatic complexity of the program using all the methods.
- List all independent paths.
- Design test cases from independent paths.

- Consider the following program segment:

```
main()
{
    int number, index;
1.  printf("Enter a number");
2.  scanf("%d", &number);
3.  index = 2;
4.  while(index <= number - 1)
5.  {
6.      if (number % index == 0)
7.      {
8.          printf("Not a prime number");
9.          break;
10.     }
11.     index++;
12. }
13. if(index == number)
14.     printf("Prime number");
15.} //end main
```


DD Graph

- The DD graph of the program is shown in the next Figure:

```
main()
{
    int number, index;
1.  printf("Enter a number");
2.  scanf("%d", &number);
3.  index = 2;
4.  while(index <= number - 1)
5.  {
6.      if (number % index == 0)
7.      {
8.          printf("Not a prime number");
9.          break;
10.     }
11.     index++;
12. }
13. if(index == number)
14.     printf("Prime number");
15.} //end main
```



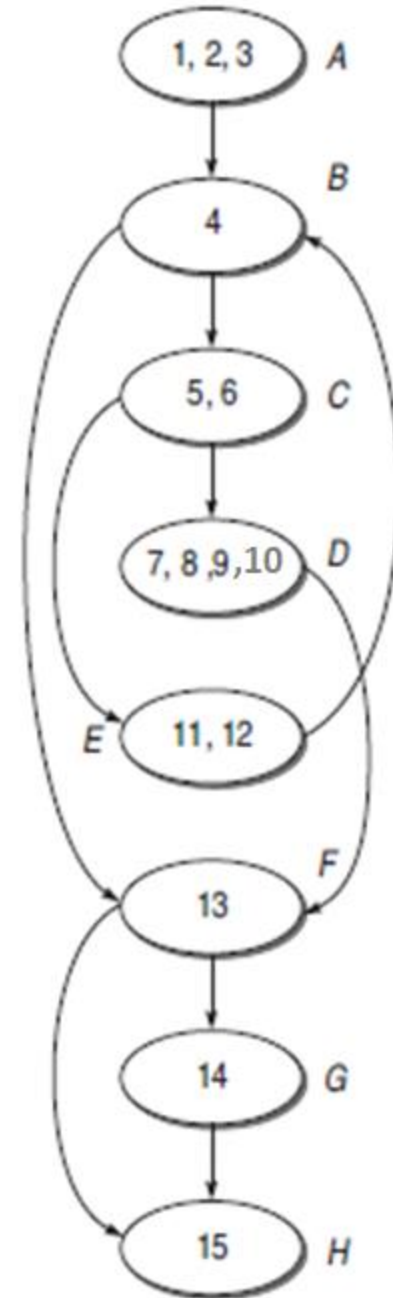
Calculating CC

Cyclomatic complexity of main()

$$\begin{aligned} V(G) &= e - n + 2 * p \\ &= 10 - 8 + 2 \\ &= 4 \end{aligned}$$

Since the cyclomatic complexity of the graph is 4, there will be 4 independent paths in the graph as shown below:

- (i) A-B-F-H
- (ii) A-B-F-G-H
- (iii) A-B-C-E-B-F-G-H
- (iv) A-B-C-D-F-H



Test Case Design from the Independent Paths

Test case ID	Input num	Expected result	Independent paths covered by test case
1	1	No output is displayed	A-B-F-H
2	2	Prime number	A-B-F-G-H
3	4	Not a prime number	A-B-C-D-F-H
4	3	Prime number	A-B-C-E-B-F-G-H

Example 2

- Draw the DD graph for the program.
- Calculate the cyclomatic complexity of the program using all the methods.
- List all independent paths.
- Design test cases from independent paths.

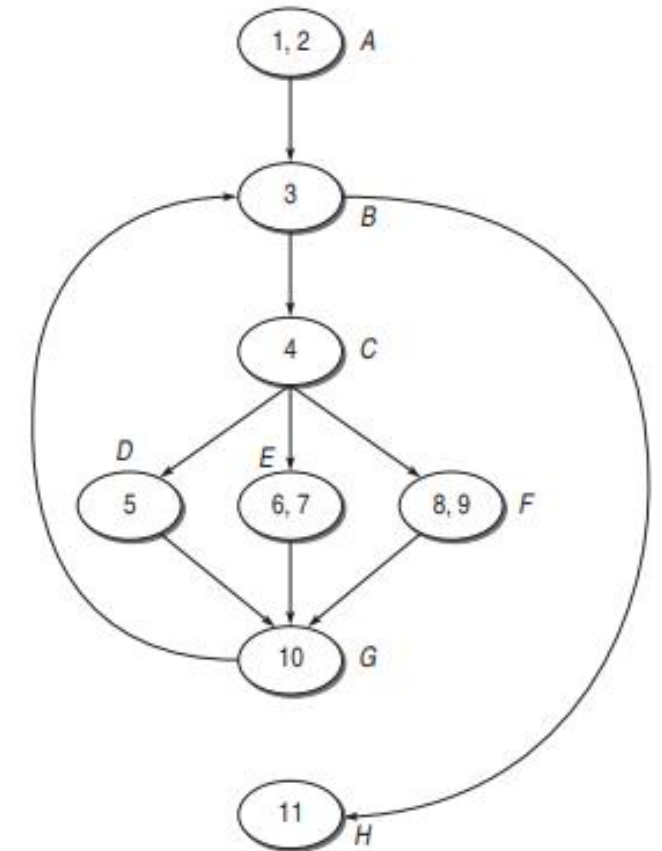
- Consider the following program segment:

```
main()
{
    char string [80];
    int index;
1.  printf("Enter the string for checking its characters");
2.  scanf("%s", string);
3.  for(index = 0; string[index] != '\0'; ++index) {
4.      if((string[index] >= '0') && (string[index] <='9'))
5.          printf("%c is a digit", string[index]);
6.      else if ((string[index] >= 'A') && (string[index]
<'Z'))||((string[index] >= 'a') && (string[index] <'z'))))
7.          printf("%c is an alphabet", string[index]);
8.      else
9.          printf("%c is a special character", string[index]);
10. }
11. }
```

DD Graph

The DD graph of the program is shown in the next Figure:

```
main()
{
    char string [80];
    int index;
1.  printf("Enter the string for checking its characters");
2.  scanf("%s", string);
3.  for(index = 0; string[index] != '\0'; ++index) {
4.      if((string[index] >= '0') && (string[index] <= '9'))
5.          printf("%c is a digit", string[index]);
6.      else if ((string[index] >= 'A') && (string[index]
7.      <'Z')) || ((string[index] >= 'a') && (string[index] < 'z'))
8.          printf("%c is an alphabet", string[index]);
9.      else
10.         printf("%c is a special character", string[index]);
11. }
```



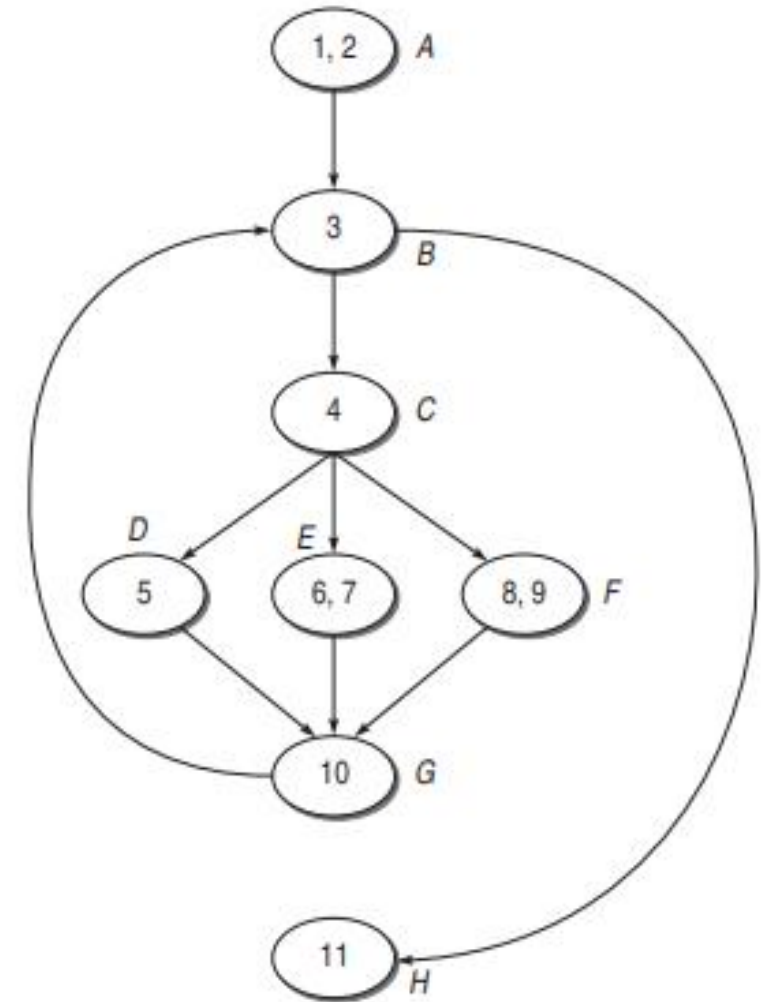
Calculating CC

Cyclomatic complexity of main()

$$\begin{aligned}V(G) &= e - n + 2 * p \\&= 10 - 8 + 2 \\&= 4\end{aligned}$$

Since the cyclomatic complexity of the graph is 4, there will be 4 independent paths in the graph as shown below:

- (i) A-B-H
- (ii) A-B-C-D-G-B-H
- (iii) A-B-C-E-G-B-H
- (iv) A-B-C-F-G-B-H



Test Case Design from the Independent Paths

Test Case ID	Input Line	Expected Output	Independent paths covered by Test case
1	0987	0 is a digit 9 is a digit 8 is a digit 7 is a digit	A-B-C-D-G-B-H A-B-H
2	AzxG	A is a alphabet z is a alphabet x is a alphabet G is a alphabet	A-B-C-E-G-B-H A-B-H
3	@#	@ is a special character # is a special character	A-B-C-F- G-B-H A-B-H

Example 3

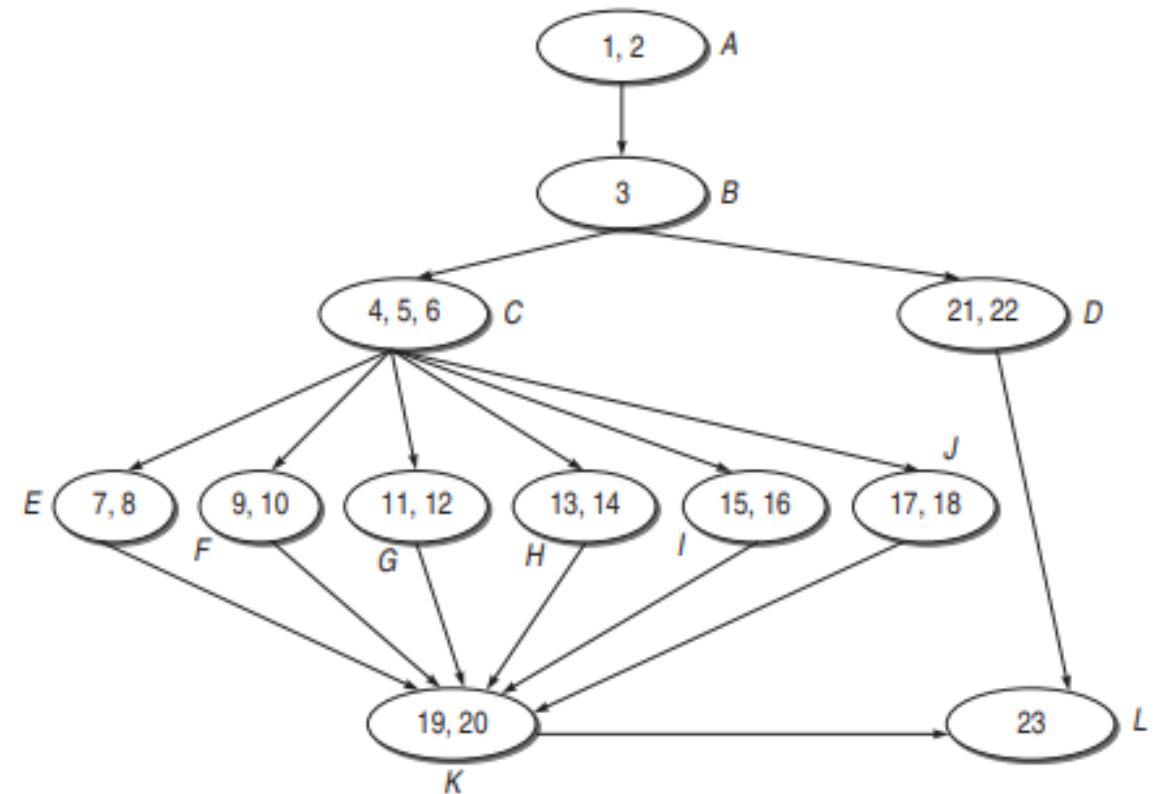
- Draw the DD graph for the program.
- Calculate the cyclomatic complexity of the program using all the methods.
- List all independent paths.
- Design test cases from independent paths.

```
main(){
    char chr;
1.    printf ("Enter the special character\n");
2.    scanf ("%c", &chr);
3.    if (chr != 48) && (chr != 49) && (chr != 50) && (chr != 51) && (chr !=
        52) && (chr != 53) && (chr != 54) && (chr != 55) && (chr != 56)
        && (chr != 57)
4.    {
5.        switch(chr)
6.        {
7.            Case '*': printf("It is a special character");
8.            break;
9.            Case '#': printf("It is a special character");
10.           break;
11.           Case '@': printf("It is a special character");
12.           break;
13.           Case '!': printf("It is a special character");
14.           break;
15.           Case '%': printf("It is a special character");
16.           break;
17.           default : printf("You have not entered a special character");
18.           break;
19.           }// end of switch
20.        } // end of If
21.    else
22.        printf("You have not entered a character");
23. } // end of main()
```


DD Graph

The DD graph of the program is shown in the next Figure:

```
main(){
    char chr;
1.    printf ("Enter the special character\n");
2.    scanf ("%c", &chr);
3.    if (chr != 48) && (chr != 49) && (chr != 50) && (chr != 51) && (chr !=
        52) && (chr != 53) && (chr != 54) && (chr != 55) && (chr != 56)
        && (chr != 57)
4.    {
5.        switch(chr)
6.        {
7.            Case '*': printf("It is a special character");
8.            break;
9.            Case '#': printf("It is a special character");
10.           break;
11.           Case '@': printf("It is a special character");
12.           break;
13.           Case '!': printf("It is a special character");
14.           break;
15.           Case '%': printf("It is a special character");
16.           break;
17.           default : printf("You have not entered a special character");
18.           break;
19.           } // end of switch
20.        } // end of If
21.    else
22.        printf("You have not entered a character");
23. } // end of main()
```



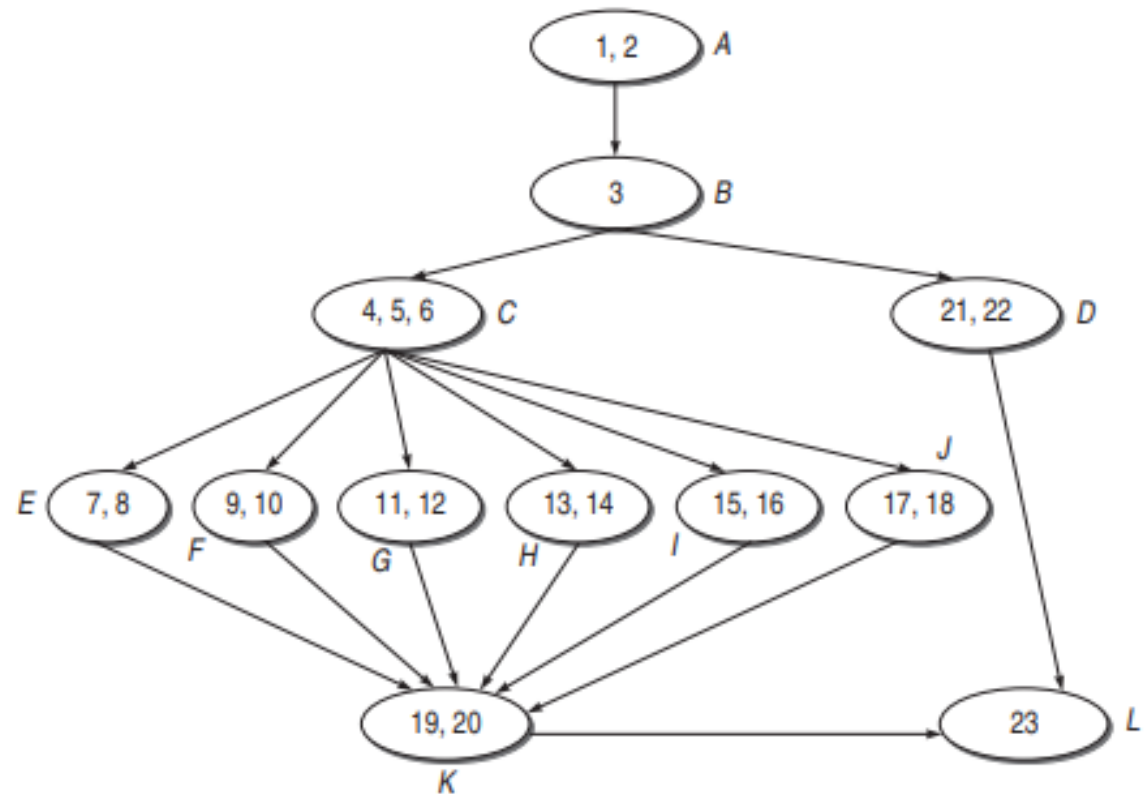
Calculating CC

Cyclomatic complexity of main()

$$\begin{aligned} V(G) &= e - n + 2p \\ &= 17 - 12 + 2 \\ &= 7 \end{aligned}$$

Since the cyclomatic complexity of the graph is 7, there will be 7 independent paths in the graph as shown below:

1. A-B-D-L
2. A-B-C-E-K-L
3. A-B-C-F-K-L
4. A-B-C-G-K-L
5. A-B-C-H-K-L
6. A-B-C-I-K-L
7. A-B-C-J-K-L



Test Case Design from the Independent Paths

Test Case ID	Input Character	Expected Output	Independent path covered by Test Case
1	(	You have not entered a character	A-B-D-L
2	*	It is a special character	A-B-C-E-K-L
3	#	It is a special character	A-B-C-F-K-L
4	@	It is a special character	A-B-C-G-K-L
5	!	It is a special character	A-B-C-H-K-L
6	%	It is a special character	A-B-C-I-K-L
7	\$	You have not entered a special character	A-B-C-J-K-L

Example 4

- Draw the DD graph for the program.
- Calculate the cyclomatic complexity of the program using all the methods.
- List all independent paths.
- Design test cases from independent paths.

- Consider the following program segment:

```
main()
{
    int number;
    int fact();
1.  clrscr();
2.  printf("Enter the number whose factorial is to be found out");
3.  scanf("%d", &number);
4.  if(number < 0)
5.      printf("Factorial cannot be defined for this number);
6.  else
7.      printf("Factorial is %d", fact(number));
8.  }

int fact(int number)
{
    int index;
1.  int product = 1;
2.  for(index=1; index<=number; index++)
3.      product = product * index;
4.  return(product);
5. }
```

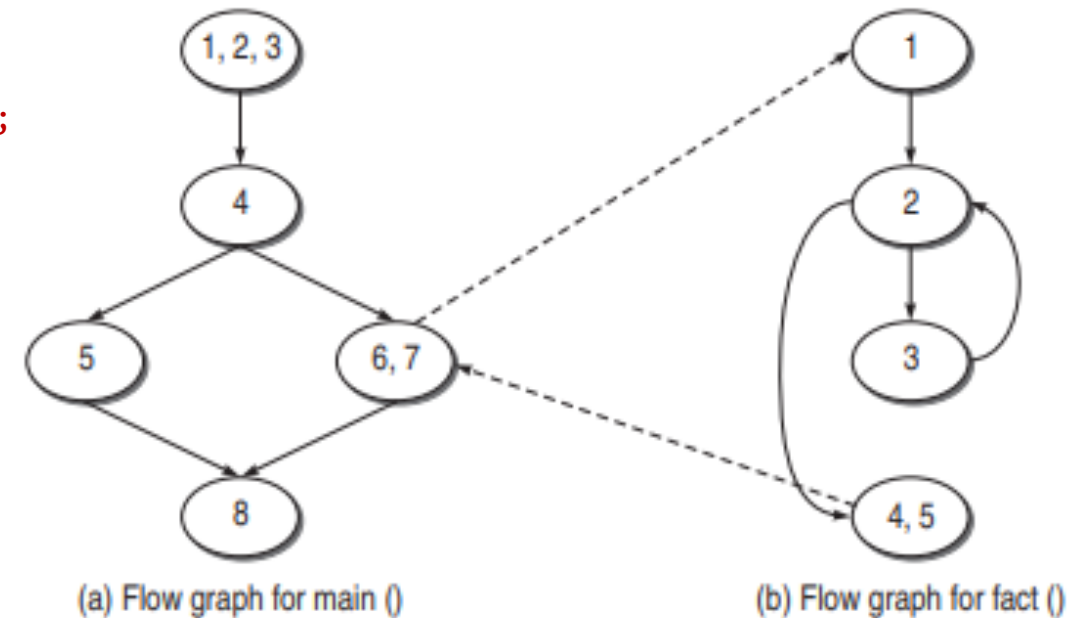
DD Graph

The DD graph of the program is shown in the next Figure:

```
main()
{
    int number;
    int fact();
    1. clrscr();
    2. printf("Enter the number whose factorial is to be found out");
    3. scanf("%d", &number);
    4. if(number < 0)
    5.     printf("Factorial cannot be defined for this number);
    6. else
    7.     printf("Factorial is %d", fact(number));
    8. }
```

```
int fact(int number)
```

```
{
    int index;
    1. int product = 1;
    2. for(index = 1; index <= number; index++)
    3.     product = product * index;
    4. return(product);
    5. }
```



Calculating CC

Cyclomatic complexity of main()

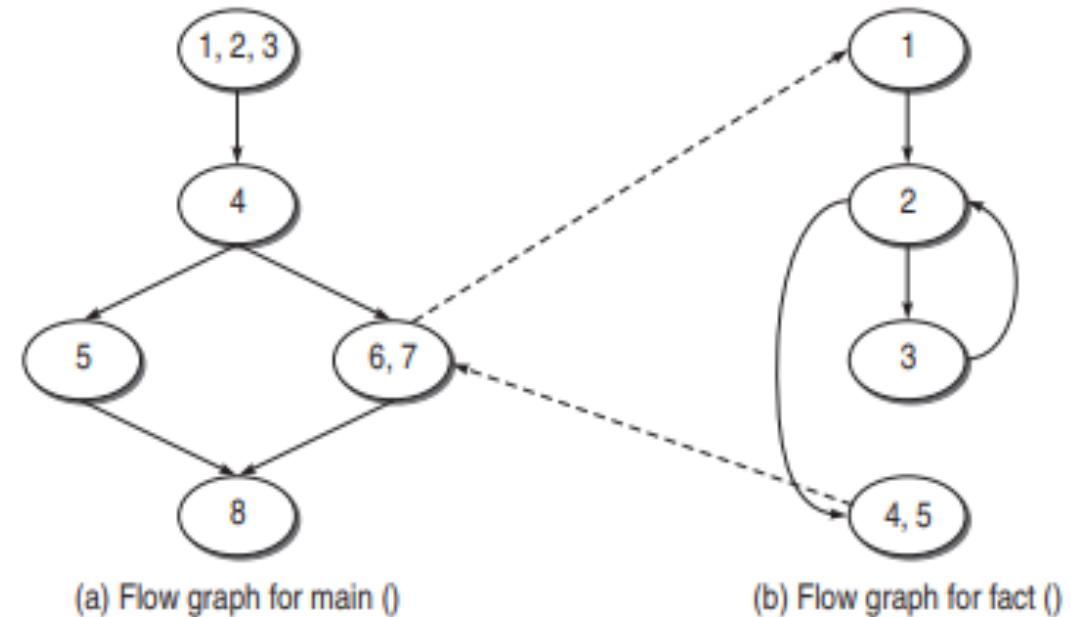
$$\begin{aligned}V(M) &= e - n + 2p \\&= 5 - 5 + 2 \\&= 2\end{aligned}$$

Cyclomatic complexity of fact()

$$\begin{aligned}V(R) &= e - n + 2p \\&= 4 - 4 + 2 \\&= 2\end{aligned}$$

Cyclomatic complexity of the whole graph considering the full program

$$\begin{aligned}(a) V(G) &= e - n + 2p \\&= 9 - 9 + 2*2 \\&= 4 \\&= V(M) + V(R)\end{aligned}$$



Halstead's Complexity

- Halstead's Software metrics are a set of measures proposed by Maurice Halstead to evaluate the complexity of a software program.
- These metrics are based on the number of distinct operators and operands in the program
- This is used to estimate the effort required to develop and maintain the program.

Notations of Halstead Metrics

- n_1 = Number of distinct operators.
- n_2 = Number of distinct operands.
- N_1 = Total number of occurrences of operators.
- N_2 = Total number of occurrences of operands.

Notations of Halstead Metrics

- Program length (N): This is the total number of operator and operand occurrences in the program.
- Vocabulary size (n): This is the total number of distinct operators and operands in the program.
- Program volume (V): This is the product of program length (N) and the logarithm of vocabulary size (n), i.e., $V = N * \log_2(n)$.
- Program level (L): This is the ratio of the number of operator occurrences to the number of operand occurrences in the program, i.e., $L = n_1/n_2$, where n_1 is the number of operator occurrences and n_2 is the number of operand occurrences.

Notations of Halstead Metrics

- Program difficulty (D): This is the ratio of the number of unique operators to the total number of operators in the program, i.e., $D = (n_1/2) * (N_2 / n_2)$.
- Program effort (E): This is the product of program volume (V) and program difficulty (D), i.e., $E = V * D$.
- Time to implement (T): This is the estimated time required to implement the program, based on the program effort (E) and a constant value that depends on the programming language and development environment.

Calculate the Metrics of the Code Snippet

```
if (x[i] < x[j])  
    {  
        Save = x[i];  
        x[i] = x[j];  
        x[j] = save;  
    }
```

Calculation of N

List of total operator and operands of the code snippet

$N_1 = 24$

$N_2 = 14$

So, Program length, $N = 38$

Operator	Operator
If	=
(	[
[	]
]	;
<	[
[	]
]	=
)	;
{	}
=	
[	
]	
;	
[	
]	

Operand
x
i
x
j
save
x
i
x
i
x
i
x
j
save

Calculation of n

List of distinct operator and operands of the code snippet

$$n_1 = 10$$

$$n_2 = 4$$

So, program vocabulary, $n = 14$

However, estimated program

length is, $\hat{N} = n_1 \log_2(n_1) + n_2$

$\log_2(n_2)$

$$= 10 \log_2(10) + 4 \log_2(4) = 41.21$$

Operator
if
(
[
]
<
)
{
=
;
}

Operand
x
i
j
save

Calculation

- Program volume (V) = $N \cdot \log_2(n)$
 $= 38 \cdot \log_2(14) = 144.68.$
- Potential Minimum Volume: The potential minimum volume V^* is defined as the volume of the most succinct program in which a problem can be coded, $V^* = (2 + n_2^*) \cdot \log_2(2 + n_2^*)$, n_2^* is the count of unique input and output parameters.

Here, $n_2^* = 6$

$$V^* = (2 + 6) \cdot \log_2(2 + 6) = 24$$

- Program level (L) = n_1 / n_2
 $= 10/4 = 2.5$

Calculation

- Program difficulty (D) = $(n_1/2) * (N_2/n_2)$
= $(10/2)*(14/4) = 17.5$
- Program effort (E) = $V*D$
= $144.68*17.5 = 2531.9$
- Program time (T) = E/β
= $2531.3/18 = 140.63$

β is a constant whose value is often considered as 18.

Topic 7.2: Black Box Testing

Black Box Testing

- Black-box testing, also called behavioral testing, focuses on the functional requirements of the software
- Different techniques
 - Graph-Based Testing Methods
 - Data Coverage Analysis
 - Boundary Value Analysis

Data Coverage Analysis

- Create columns to use in the data coverage analysis

Create columns that map to source data columns to use in the analysis. You can create columns that use source data as is or in ranges of data values. You can also create columns that map data values across specific groups of values.
- Run the data coverage task and view the results of the analysis on the data coverage page

You can run the data coverage task and view the results in the data coverage page. You can edit the results view to select and view specific cells in the results.
- Edit the parameters used in the analysis

You can edit the parameters including the columns and filters applied and view updated analysis results in the data coverage page.
- Update data values in the source data

You can update data values across cells that you analyze. For example, based on the data coverage analysis results, a cell contains data values below the minimum threshold that you set. You can update data in other cells to create the minimum required data values in the cell.

Decision Table

- Decision table is a useful method to represent the information in a tabular method. It has the specialty to consider complex combinations of input conditions and resulting actions.
- Decision tables obtain their power from logical expressions. Each operand or variable in a logical expression takes on the value, TRUE or FALSE.

Formation Of Decision Table

ENTRY

Condition Stub		Rule 1	Rule 2	Rule 3	Rule 4	...
	C1	True	True	False	I	
	C2	False	True	False	True	
	C3	True	True	True	I	
Action Stub	A1		X			
	A2	X			X	
	A3			X		

Decision table structure

Decision Table Structure

- Condition Stub
 - It is a list of input conditions for which the complex combination is made.
- Action Stub
 - It is a list of resulting actions which will be performed if a combination of input condition is satisfied.
- Condition Entry
 - It is a specific entry in the table corresponding to input conditions mentioned in the condition stub. When we enter TRUE or FALSE for all input conditions for a particular combination, then it is called a Rule.
 - When the condition entry takes only two values—TRUE or FALSE, then it is called Limited Entry Decision Table.

Decision Table Structure

- When the condition entry takes several values, then it is called Extended Entry Decision Table.
- In limited entry decision table, condition entry, which has no effect whether it is True or False, is called a Don't-Care state or immaterial state.
- Action Entry
 - It is the entry in the table for the resulting action to be performed when one rule (which is a combination of input condition) is satisfied. 'X' denotes the action entry in the table.

Test Case Design Using Decision Table

- For designing test cases from a decision table, following interpretations should be done:
 - Interpret condition stubs as the inputs for the test case.
 - Interpret action stubs as the expected output for the test case.
 - Rule, which is the combination of input conditions, becomes the test case itself.
 - If there are **k** rules over n binary conditions, there are at least **k** test cases and at the most **2^n** test cases.

Example

A program calculates the total salary of an employee with the conditions that if the working hours are less than or equal to 48, then give normal salary. The hours over 48 on normal working days are calculated at the rate of 1.25 of the salary. However, on holidays or Sundays, the hours are calculated at the rate of 2.00 times of the salary. Design test cases using decision table testing.

Example: Solution

- The decision table for the program is shown below:

ENTRY				
		Rule 1	Rule 2	Rule3
Condition Stub	C1: Working hours > 48	I	F	T
	C2: Holidays or Sundays	T	F	F
Action Stub	A1: Normal salary		X	
	A2: 1.25 of salary			X
	A3: 2.00 of salary	X		

The test cases derived from the decision table are given below:

Test Case ID	Working Hour	Day	Expected Result
1	48	Monday	Normal Salary
2	50	Tuesday	1.25 of salary
3	52	Sunday	2.00 of salary

Example 2

A wholesaler has three commodities to sell and has three types of customers. Discount is given as per the following procedure:

- (i) For DGS & D orders, 10% discount is given irrespective of the value of the order.
- (ii) For orders of more than Rs 50,000, agents get a discount of 15% and the retailer gets a discount of 10%.
- (iii) For orders of Rs 20,000 or more and up to Rs 50,000, agents get 12% and the retailer gets 8% discount.
- (iv) For orders of less than Rs 20,000, agents get 8% and the retailer gets 5% discount.

The above rules do not apply to the furniture items wherein a flat rate of 10% discount is admissible to all customers irrespective of the value of the order.

Design test cases for this system using decision table testing.

Example 2: Solution

		ENTRY							
		R1	R2	R3	R4	R5	R6	R7	R8
Condition Stub	C1: DGS & D	T	F	F	F	F	F	F	I
	C2: Agent	F	T	F	T	F	T	F	I
	C3: Retailer	F	F	T	F	T	F	T	I
	C4: Order > 50,000	I	T	T	F	F	F	F	I
	C5: Order ≥ 20000 to < 50,000	I	F	F	T	T	F	F	I
	C6: Order < 20,000	I	F	F	F	F	T	T	I
	C7: Furniture	F	F	F	F	F	F	F	T
Action Stub	A1: Discount of 5%							X	
	A2: Discount of 8%					X	X		
	A3: Discount of 10%	X		X					X
	A4: Discount of 12%				X				
	A5: Discount of 15%		X						

Example 2: Solution

The test cases derived from the decision table are given below:

Test Case ID	Type of Customer	Product Furniture?	Order Value (Rs)	Expected Result
1	DGS & D	No	51,000	10% Discount
2	Agent	No	52,000	15% Discount
3	Retailer	No	53,000	10% Discount
4	Agent	No	23,000	12% Discount
5	Retailer	No	27,000	8% Discount
6	Agent	No	15,000	8% Discount
7	Retailer	No	18,000	5% Discount
8	Agent	Yes	34,000	10% Discount

Example 3

A university is admitting students in a professional course subject to the following conditions:

- (a) Marks in Java ≥ 70
- (b) Marks in C++ ≥ 60
- (c) Marks in OOAD ≥ 60
- (d) Total in all three subjects ≥ 220 OR Total in Java and C++ ≥ 150

If the aggregate mark of an eligible candidate is more than 240, he will be eligible for scholarship course, otherwise he will be eligible for normal course. The program reads the marks in the three subjects and generates the following outputs:

- (i) Not eligible
- (ii) Eligible for scholarship course
- (iii) Eligible for normal course

Design test cases for this program using decision table testing.

Example 3: Solution

	ENTRY							
	R1	R2	R3	R4	R5	R6	R7	R8
C1: marks in Java ≥ 70	T	T	T	F	I	I	I	T
C2: marks in C++ ≥ 60	T	T	T	I	F	I	I	T
C3: marks in OOAD ≥ 60	T	T	T	I	I	F	I	T
C4: Total in three subjects ≥ 220	T	F	T	I	I	I	F	T
C5: Total in Java & C++ ≥ 150	F	T	I	I	I	I	F	T
C6: Aggregate marks > 240	F	F	T	I	I	I	I	F
A1: Eligible for normal course	X	X						X
A2: Eligible for scholarship course			X					
A3: Not eligible				X	X	X	X	

Example 3: Solution

The test cases derived from the decision table are given below:

Test Case ID	Java	C++	OOAD	Aggregate Marks	Expected Output
1	75	75	70	220	Eligible for normal course
2	75	75	60	210	Eligible for normal course
3	75	74	93	242	Eligible for scholarship course
4	78	45	78	201	Not eligible
5	68	78	80	226	Not eligible
6	70	72	70	212	Not eligible
7	80	80	50	210	Not eligible
8	75	75	75	225	Eligible for normal course

Example 4 (Immaterial Cases)

- Consider Example 1 once again whose decision table is shown below with immaterial cases.

ENTRY				
		Rule 1	Rule 2	Rule 3
Condition Stub	C1: Working hours > 48	I	F	T
	C2: Holidays or Sundays	T	F	F
Action Stub	A1: Normal salary		X	
	A2: 1.25 of salary			X
	A3: 2.00 of salary	X		

Example 4: Solution

- The immaterial test case in Rule 1 of the above table can be expanded by taking both T and F values of C1. The expanded decision table is shown below:

ENTRY					
		Rule 1-1	Rule 1-2	Rule 2	Rule 3
Condition Stub	C1: Working hours > 48	F	T	F	T
	C2: Holidays or Sundays	T	T	F	F
Action Stub	A1: Normal salary			X	
	A2: 1.25 of salary				X
	A3: 2.00 of salary	X	X		

Example 4: Solution

The test cases derived from the decision table are given below:

Test Case ID	Working Hour	Day	Expected Result
1	48	Monday	Normal salary
2	50	Tuesday	1.25 of salary
3	52	Sunday	2.00 of salary
4	30	Sunday	2.00 of salary

Boundary Value Analysis (BVA)

- It is considered a technique that uncovers the bugs at the boundary of input values. Here, boundary means the maximum or minimum value taken by the input domain.
- It is an effective test case design that requires test cases to be designed such that they maximize the probability of finding errors.
- With the experience of testing team, it has been observed that test cases designed with boundary input values have a high chance to find errors. It means that most of the failures crop up due to boundary values.

BVA: Example

If **A** is an integer **between 10 and 255**, then boundary checking can be on **10(9,10,11)** and on **255(256,255,254)**. Similarly, **B** is another integer variable **between 10 and 100**, then boundary checking can be on **10(9,10,11)** and **100(99,100,101)**



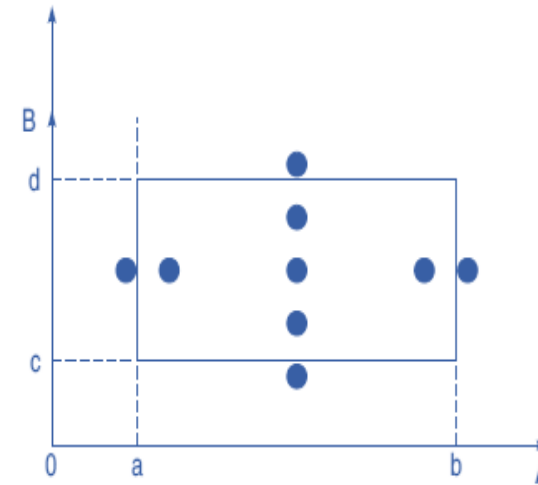
Boundary Value Checking (BVC)

- In this method, the test cases are designed by holding one variable at its extreme value and other variables at their nominal values in the input domain.
- The variable at its extreme value can be selected at:
 - Minimum value (Min)
 - Value just above the minimum value (Min+)
 - Maximum value (Max)
 - Value just below the maximum value (Max-)

BVC: Example

Let us take the example of two variables, A and B. If we consider all the above combinations with nominal values, then following test cases can be designed:

- | | |
|-------------------------------------|--------------------------------------|
| 1. $A_{\text{nom}}, B_{\text{min}}$ | 2. $A_{\text{nom}}, B_{\text{min}+}$ |
| 3. $A_{\text{nom}}, B_{\text{max}}$ | 4. $A_{\text{nom}}, B_{\text{max}-}$ |
| 5. $A_{\text{min}}, B_{\text{nom}}$ | 6. $A_{\text{min}+}, B_{\text{nom}}$ |
| 7. $A_{\text{max}}, B_{\text{nom}}$ | 8. $A_{\text{max}-}, B_{\text{nom}}$ |
| 9. $A_{\text{nom}}, B_{\text{nom}}$ | |



It can be generalized that for n variables in a module, **$4n + 1$** test cases can be designed with boundary value checking method.

Robustness Testing Method

- The idea of BVC can be extended such that boundary values are exceeded as:
 - A value just greater than the Maximum value (Max+)
 - A value just less than Minimum value (Min-)
- When test cases are designed considering the above points in addition to BVC, it is called robustness testing.

Robustness Testing Method: Example

- Let us take the previous example again. Add the following test cases to the list of 9 test cases designed in BVC:
 - 10. $A_{\text{max+}}, B_{\text{nom}}$
 - 11. $A_{\text{min-}}, B_{\text{nom}}$
 - 12. $A_{\text{nom}}, B_{\text{max+}}$
 - 13. $A_{\text{nom}}, B_{\text{min-}}$
- It can be generalized that for **n** input variables in a module, **6n + 1** test cases can be designed with robustness testing.

Worst-case Testing Method

- We can again extend the concept of BVC by assuming more than one variable on the boundary. It is called worst-case testing method.
- following test cases to the list of 9 test cases designed in BVC as:

10. $A_{\min}, B_{\min}$	11. $A_{\min+}, B_{\min}$
12. $A_{\min}, B_{\min+}$	13. $A_{\min+}, B_{\min+}$
14. $A_{\max}, B_{\min}$	15. $A_{\max-}, B_{\min}$
16. $A_{\max}, B_{\min+}$	17. $A_{\max-}, B_{\min+}$
18. $A_{\min}, B_{\max}$	19. $A_{\min+}, B_{\max}$
20. $A_{\min}, B_{\max-}$	21. $A_{\min+}, B_{\max-}$
22. $A_{\max}, B_{\max}$	23. $A_{\max-}, B_{\max}$
24. $A_{\max}, B_{\max-}$	25. $A_{\max-}, B_{\max-}$
- It can be generalized that for n input variables in a module, 5^n test cases can be designed with worst-case testing.

Example 1

A program reads an integer number within the range [1,100] and determines whether it is a prime number or not. Design test cases for this program using **BVC**, **robust testing**, and **worst-case testing** methods.

Example 1 : Solution

- **Test cases using BVC:** Since there is one variable, the total number of test cases will be $4n + 1 = 5$. In our example, the set of minimum and maximum values is shown below:

Min value = 1
Min ⁺ value = 2
Max value = 100
Max ⁻ value = 99
Nominal value = 50–55

Example 1 : Solution

Using these values, test cases can be designed as shown below:

Test Case ID	Integer Variable	Expected Output
1	1	Not a prime number
2	2	Prime number
3	100	Not a prime number
4	99	Not a prime number
5	53	Prime number

Example 1 : Solution

- **Test cases using robust testing:** Since there is one variable, the total number of test cases will be $6n + 1 = 7$. The set of boundary values is shown below:

Min value = 1
Min ⁻ value = 0
Min ⁺ value = 2
Max value = 100
Max ⁻ value = 99
Max ⁺ value = 101
Nominal value = 50–55

Example 1 : Solution

Using these values, test cases can be designed as shown below:

Test Case ID	Integer Variable	Expected Output
1	0	Invalid input
2	1	Not a prime number
3	2	Prime number
4	100	Not a prime number
5	99	Not a prime number
6	101	Invalid input
7	53	Prime number

Example 1 : Solution

- **Test cases using worst-case testing:** Since there is one variable, the total number of test cases will be $5^n = 5$. Therefore, the number of test cases will be same as BVC.

Example 2

A program computes a^b where a lies in the range $[1,10]$ and b within $[1,5]$. Design test cases for this program using **BVC, robust testing, and worst-case** testing methods.

Example 2 : Solution

- **Test cases using BVC:**

Since there are two variables, a and b, the total number of test cases will be $4n + 1 = 9$. The set of boundary values is shown below:

	a	b
Min value	1	1
Min ⁺ value	2	2
Max value	10	5
Max ⁻ value	9	4
Nominal value	5	3

Example 2 : Solution

Using these values, test cases can be designed as shown below:

Test Case ID	a	b	Expected Output
1	1	3	1
2	2	3	8
3	10	3	1000
4	9	3	729
5	5	1	5
6	5	2	25
7	5	4	625
8	5	5	3125
9	5	3	125

Example 2 : Solution

- **Test cases using robust testing:** Since there are two variables, a and b, the total number of test cases will be $6n + 1 = 13$. The set of boundary values is shown below:

	a	b
Min value	1	1
Min ⁻ value	0	0
Min ⁺ value	2	2
Max value	10	5
Max ⁺ value	11	6
Max ⁻ value	9	4
Nominal value	5	3

Example 2 : Solution

Using these values, test cases can be designed as shown:

Test Case ID	a	b	Expected output
1	0	3	Invalid input
2	1	3	1
3	2	3	8
4	10	3	1000
5	11	3	Invalid input
6	9	3	729
7	5	0	Invalid input
8	5	1	5
9	5	2	25
10	5	4	625
11	5	5	3125
12	5	6	Invalid input
13	5	3	125

Example 2 : Solution

- **Test cases using worst-case testing:** Since there are two variables, a and b, the total number of test cases will be $5^n = 25$.

	a	b
Min value	1	1
Min ⁺ value	2	2
Max value	10	5
Max ⁻ value	9	4
Nominal value	5	3

Example 2 : Solution

Using these values, test cases can be designed as shown:

Test Case ID	a	b	Expected Output
1	1	1	1
2	1	2	1
3	1	3	1
4	1	4	1
5	1	5	1
6	2	1	2
7	2	2	4
8	2	3	8
9	2	4	16
10	2	5	32
11	5	1	5
12	5	2	25
13	5	3	125
14	5	4	625
15	5	5	3125
16	9	1	9
17	9	2	81
18	9	3	729
19	9	4	6561
20	9	5	59049
21	10	1	10
22	10	2	100
23	10	3	1000
24	10	4	10000
25	10	5	100000

Example 3

A program reads three numbers, A, B, and C, within the range [1, 50] and prints the largest number. Design test cases for this program using BVC, robust testing, and worst-case testing methods.

Example 3 : Solution

- **Test cases using BVC:**

Since there is one variable, the total number of test cases will be $4n + 1 = 5$. In our example, the set of minimum and maximum values is shown below:

Min value = 1
Min ⁺ value = 2
Max value = 50
Max ⁻ value = 49
Nominal value = 25–30

Example 3 : Solution

Using these values, test cases can be designed as shown below:

Test Case ID	A	B	C	Expected Output
1	1	25	27	C is largest
2	2	25	28	C is largest
3	49	25	25	A is largest
4	50	25	29	A is largest
5	25	1	30	C is largest
6	25	2	26	C is largest
7	25	49	27	B is largest
8	25	50	28	B is largest
9	25	28	1	B is largest
10	25	27	2	B is largest
11	25	26	49	C is largest
12	25	26	50	C is largest
13	25	25	25	Three are equal

Example 3 : Solution

- **Test cases robust testing:** Since there are three variables, A, B, and C, the total number of test cases will be $6n + 1 = 19$.

Min value = 1

Min⁻ value = 0

Min⁺ value = 2

Max value = 50

Max⁺ value = 51

Max⁻ value = 49

Nominal value = 25–30

Example 3 : Solution

Using these values, test cases can be designed as shown:

Test Case ID	A	B	C	Expected Output
1	0	25	27	Invalid input
2	1	25	27	C is largest
3	2	25	28	C is largest
4	49	25	25	A is largest
5	50	25	29	A is largest
6	51	27	25	Invalid input
7	25	0	26	Invalid input
8	25	1	30	C is largest
9	25	2	26	C is largest
10	25	49	27	B is largest
11	25	50	28	B is largest
12	26	51	25	Invalid input
13	25	25	0	Invalid input
14	25	28	1	B is largest
15	25	27	2	B is largest
16	25	26	49	C is largest
17	25	26	50	C is largest
18	25	29	51	Invalid input
19	25	25	25	Three are equal

4 PILLARS

Encapsulation

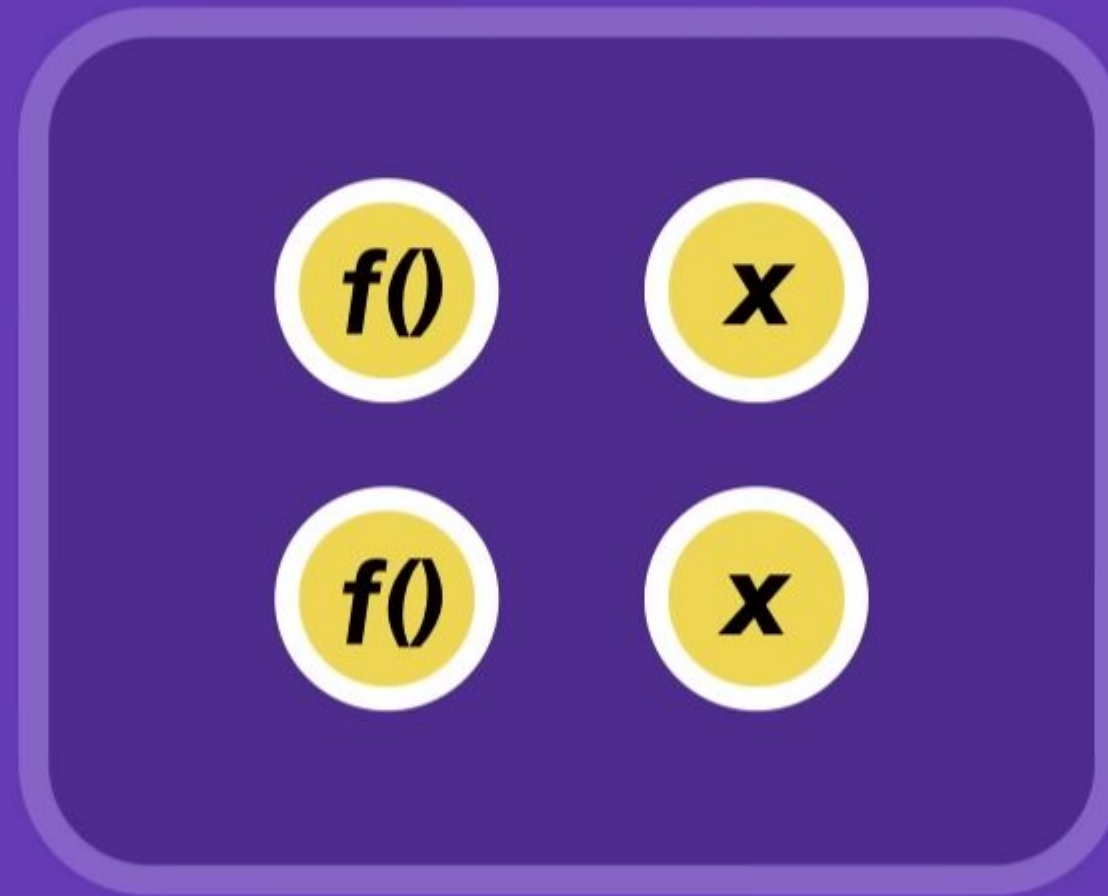
Abstraction

Inheritance

Polymorphism

PROCEDURAL PROGRAMMING

Program

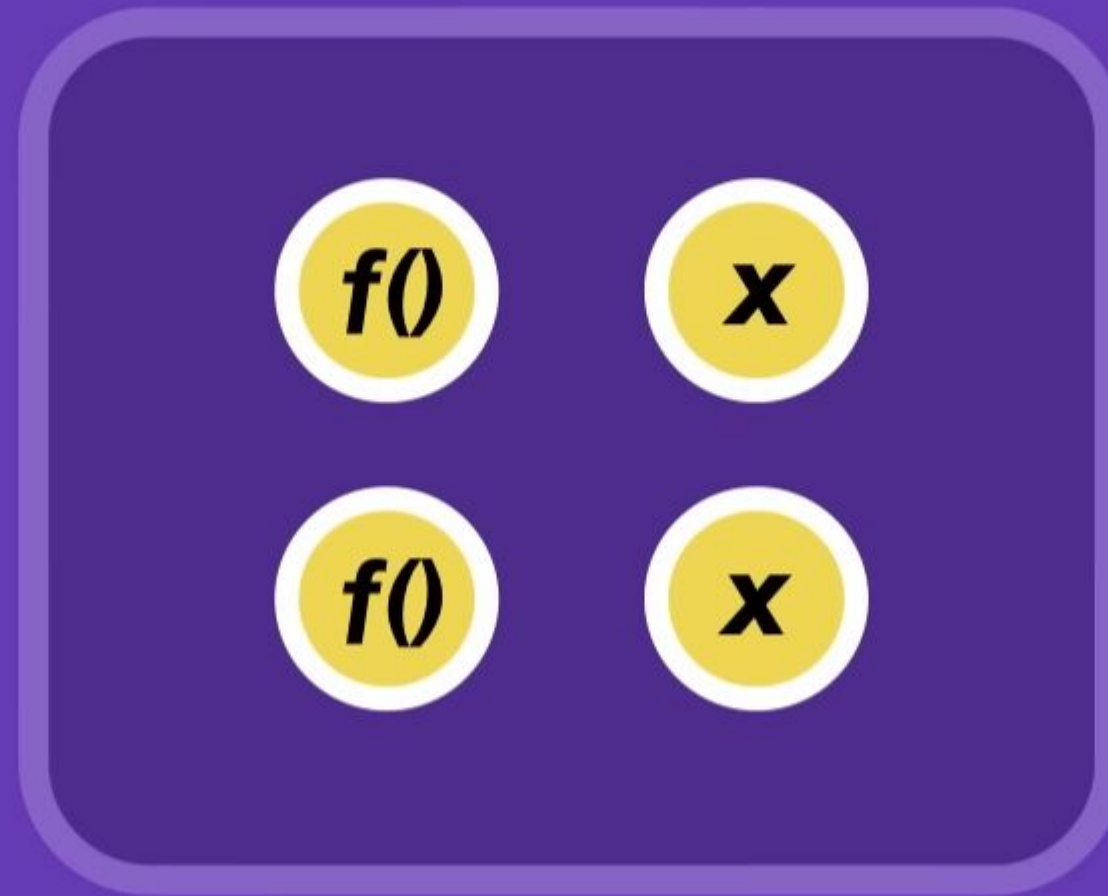


PROCEDURAL PROGRAMMING



Structured Programming?

Program



PROCEDURAL PROGRAMMING



Structured Programming?

Procedural Programming is a subset of structured programming, where programs are divided into several functions / procedures. So all procedural programs are structured programs, but not all structured programs are procedural programs.

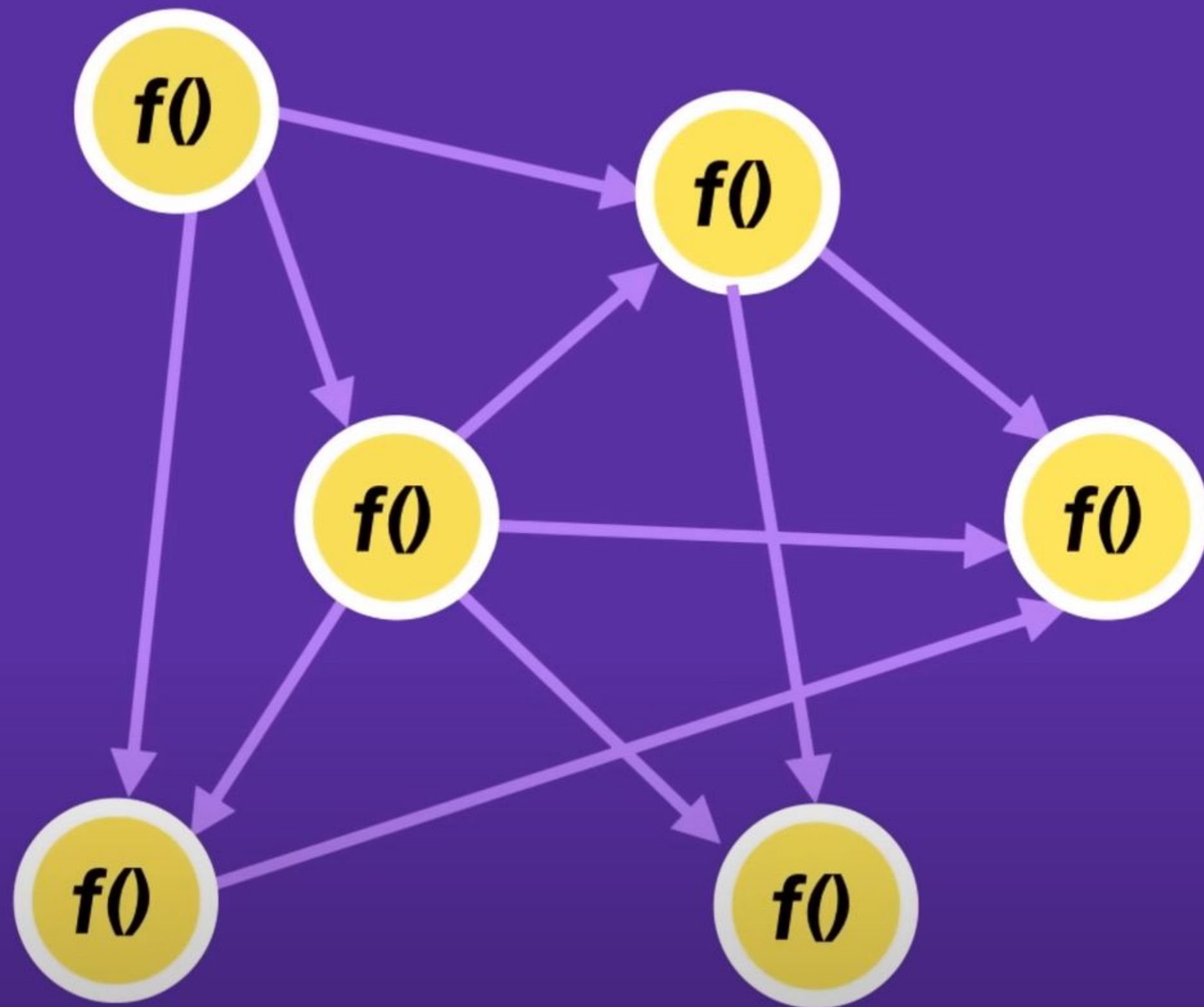
Program

f()

x

f()

x





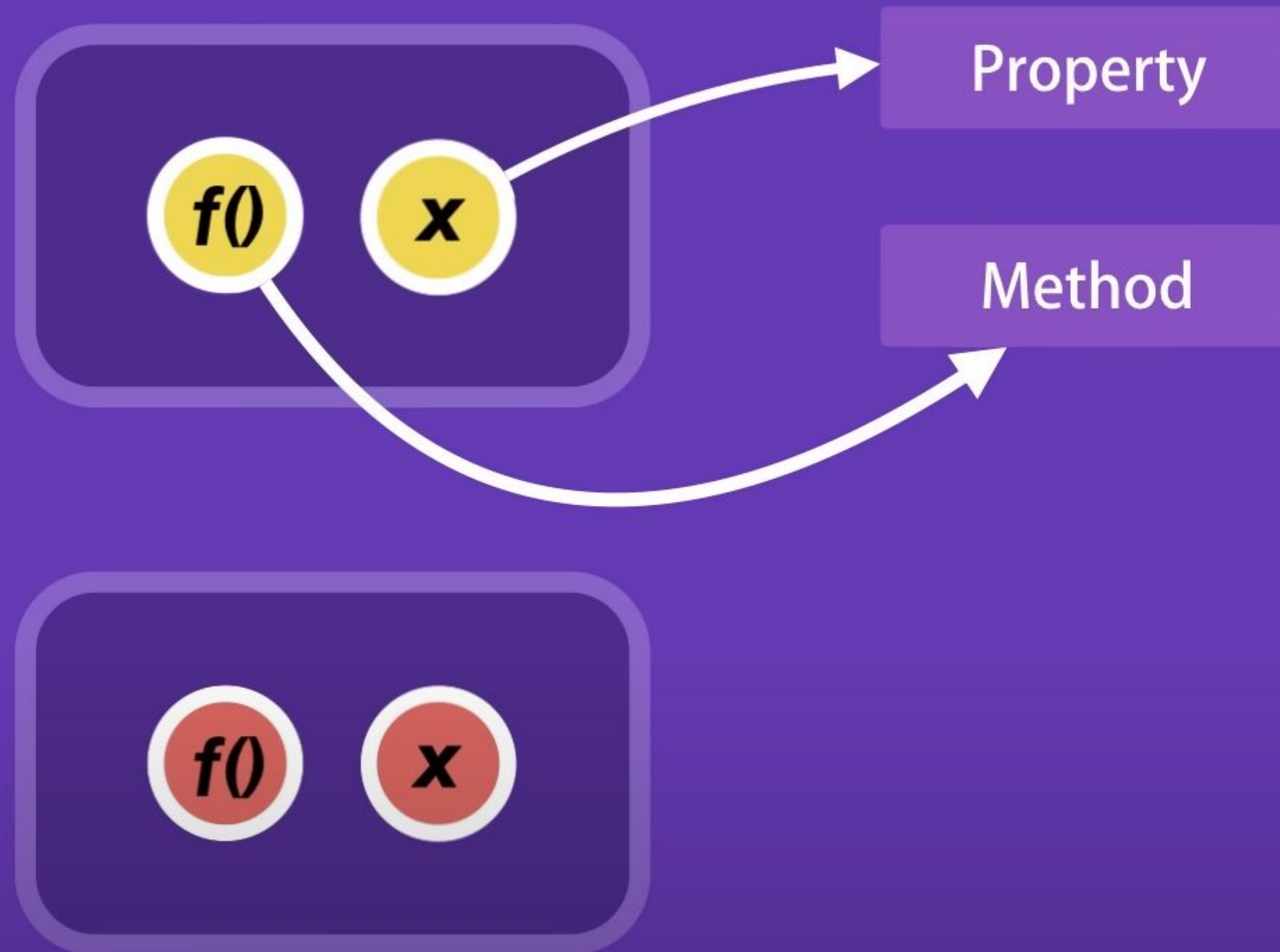
Object-oriented Programming

f()

x

f()

x

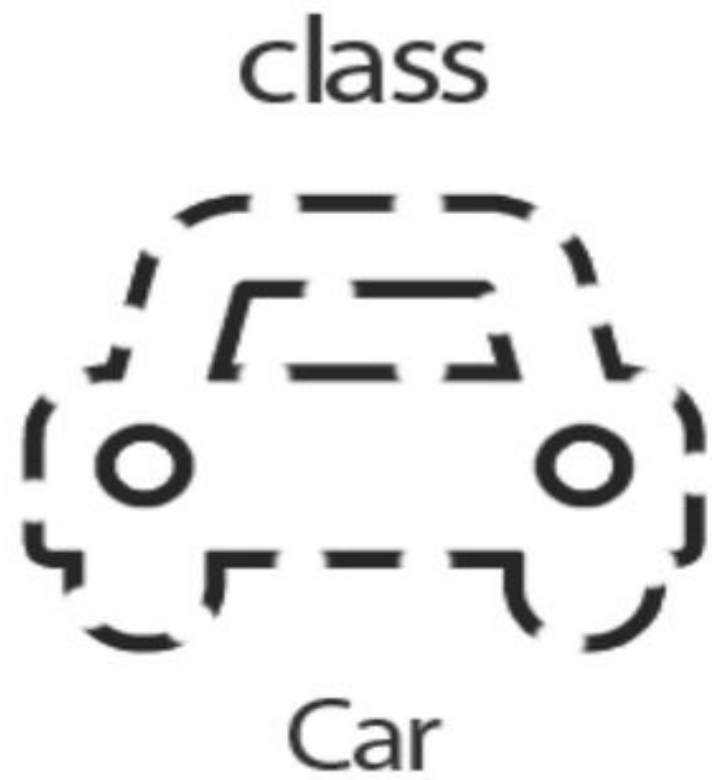


Car

make
model
color

start()
stop()
move()

Class vs Object



Audi

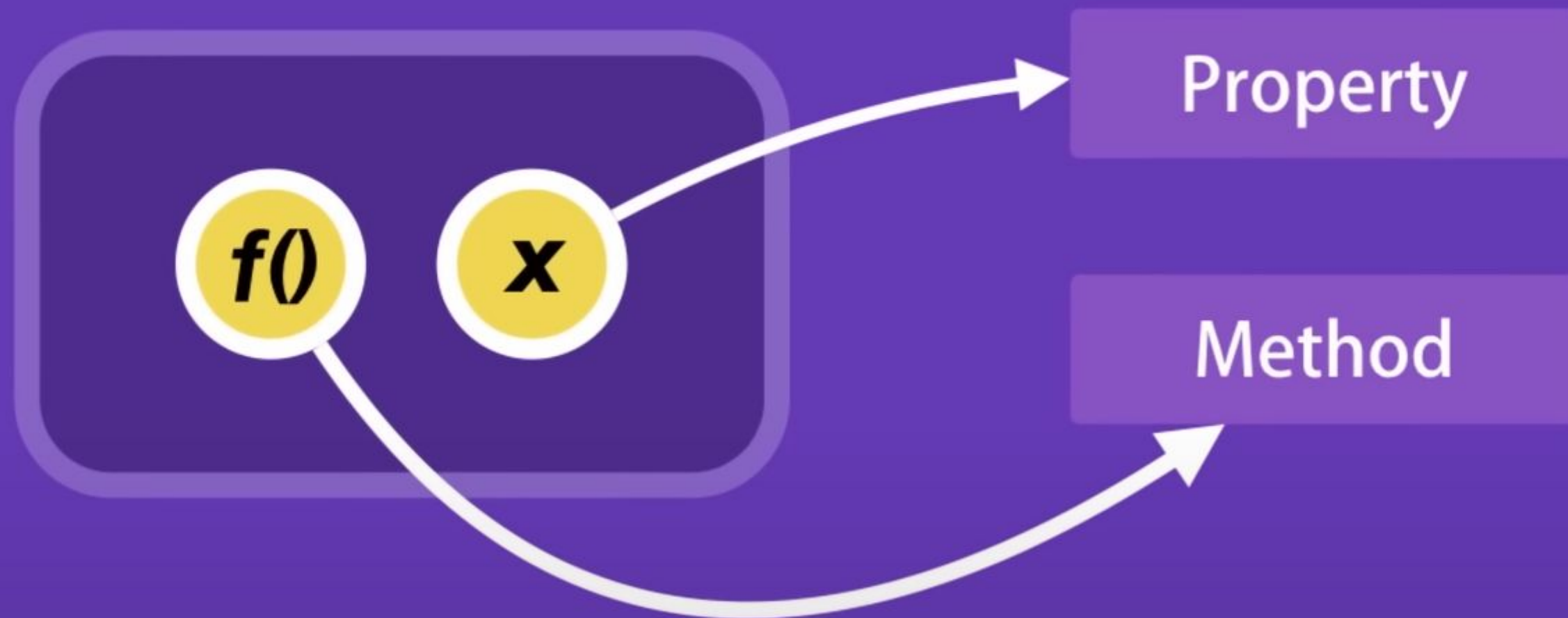


Nissan



Volvo

ENCAPSULATION



ENCAPSULATION

class

{

data members

+

methods (behavior)

}

E
N
C
A
P
S
U
L
A
T
I
O
N

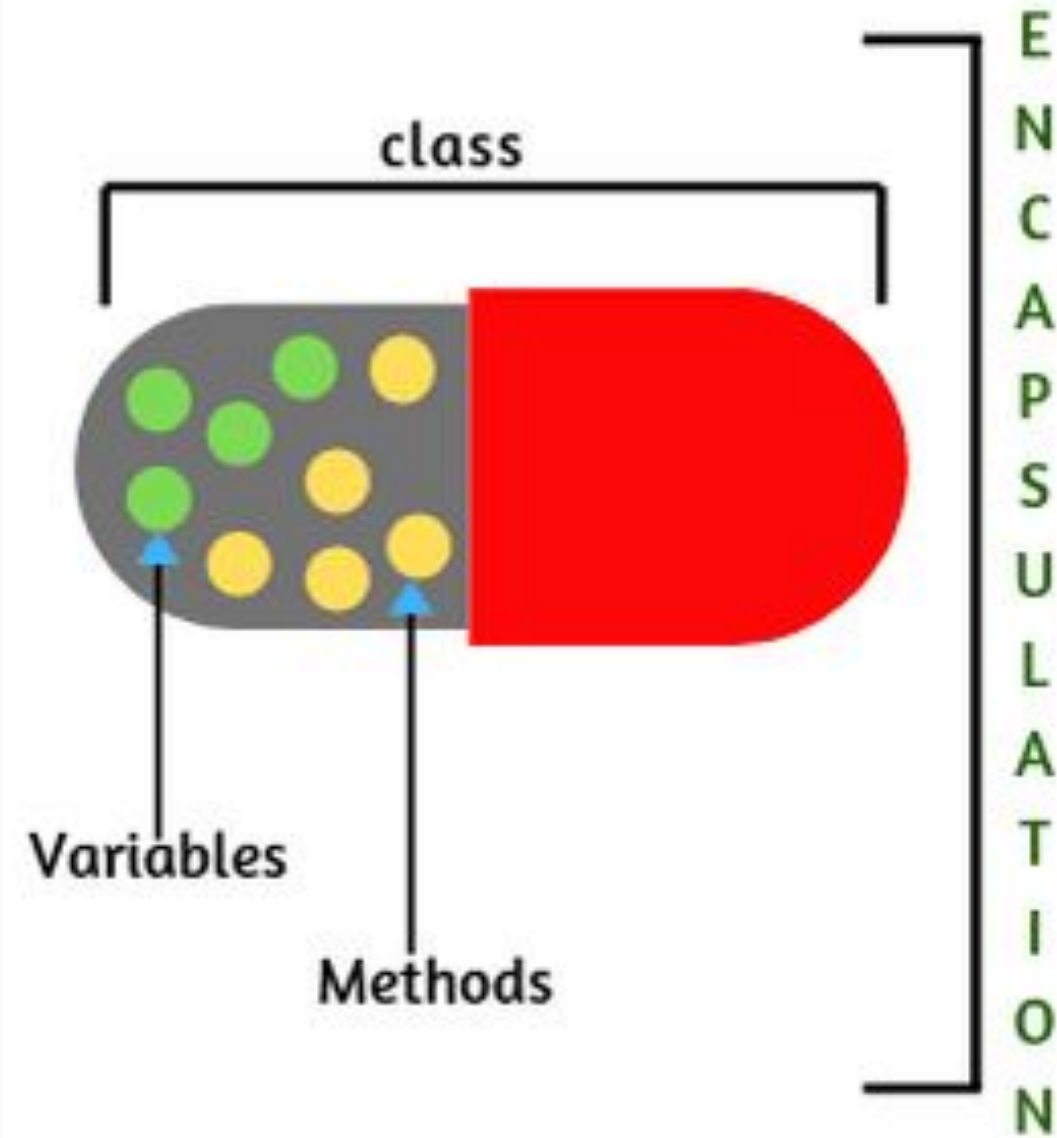


Fig: Encapsulation

Structured Programming

```
#include <stdio.h>

int main() {
    int baseSalary = 30000;
    int overtime = 10;
    int rate = 20;

    int overtimePay = overtime * rate;
    int totalWage = baseSalary + overtimePay;

    printf("Total Wage: %d\n", totalWage);

    return 0;
}
```

Procedural Programming

```
#include <stdio.h>

// Function to calculate overtime pay separately
int calculateOvertimePay(int overtime, int rate) {
    return overtime * rate;
}

// Function to calculate total wage
int getWage(int baseSalary, int overtime, int rate) {
    return baseSalary + calculateOvertimePay(overtime, rate);
}

int main() {
    int baseSalary = 30000;
    int overtime = 10;
    int rate = 20;

    int wage = getWage(baseSalary, overtime, rate);
    printf("Total Wage: %d\n", wage);

    return 0;
}
```

Object Oriented Programming

```
class Employee {  
    private int baseSalary;  
    private int overtime;  
    private int rate;  
  
    // Constructor  
    public Employee(int baseSalary, int overtime, int rate) {  
        this.baseSalary = baseSalary;  
        this.overtime = overtime;  
        this.rate = rate;  
    }  
  
    // Method to calculate overtime pay separately  
    private int calculateOvertimePay() {  
        return overtime * rate;  
    }  
  
    // Method to calculate total wage  
    public int getWage() {  
        return baseSalary + calculateOvertimePay();  
    }  
  
    // Main method  
    public static void main(String[] args) {  
        Employee emp = new Employee(30000, 10, 20);  
        System.out.println("Total Wage: " + emp.getWage());  
    }  
}
```

Object Oriented Programming

```
class Employee {  
    private int baseSalary;  
    private int overtime;  
    private int rate;  
  
    // Constructor  
    public Employee(int baseSalary, int overtime, int rate) {  
        this.baseSalary = baseSalary;  
        this.overtime = overtime;  
        this.rate = rate;  
    }  
  
    // Method to calculate overtime pay separately  
    private int calculateOvertimePay() {   
        return overtime * rate;  
    }  
  
    // Method to calculate total wage  
    public int getWage() {   
        return baseSalary + calculateOvertimePay();  
    }  
  
    // Main method  
    public static void main(String[] args) {  
        Employee emp = new Employee(30000, 10, 20);  
        System.out.println("Total Wage: " + emp.getWage());  
    }  
}
```

Procedural Programming

```
#include <stdio.h>

// Function to calculate overtime pay separately
int calculateOvertimePay(int overtime, int rate) {
    return overtime * rate;
}

// Function to calculate total wage
int getWage(int baseSalary, int overtime, int rate) {
    return baseSalary + calculateOvertimePay(overtime, rate);
}

int main() {
    int baseSalary = 30000;
    int overtime = 10;
    int rate = 20;

    int wage = getWage(baseSalary, overtime, rate);
    printf("Total Wage: %d\n", wage);

    return 0;
}
```


Object Oriented Programming

```
class Employee {  
    private int baseSalary;  
    private int overtime;  
    private int rate;  
  
    // Constructor  
    public Employee(int baseSalary, int overtime, int rate) {  
        this.baseSalary = baseSalary;  
        this.overtime = overtime;  
        this.rate = rate;  
    }  
  
    // Method to calculate overtime pay separately  
    private int calculateOvertimePay() {   
        return overtime * rate;  
    }  
  
    // Method to calculate total wage  
    public int getWage() {   
        return baseSalary + calculateOvertimePay();  
    }  
  
    // Main method  
    public static void main(String[] args) {  
        Employee emp = new Employee(30000, 10, 20);  
        System.out.println("Total Wage: " + emp.getWage());  
    }  
}
```

Object Oriented Programming

```
class Employee {  
    private int baseSalary;  
    private int overtime;  
    private int rate;  
  
    // Constructor  
    public Employee(int baseSalary, int overtime, int rate) {  
        this.baseSalary = baseSalary;  
        this.overtime = overtime;  
        this.rate = rate;  
    }  
  
    // Method to calculate overtime pay separately  
    private int calculateOvertimePay() {  
        return overtime * rate;  
    }  
  
    // Method to calculate total wage  
    public int getWage() {  
        return baseSalary + calculateOvertimePay();  
    }  
  
    // Main method  
    public static void main(String[] args) {  
        Employee emp = new Employee(30000, 10, 20);  
        System.out.println("Total Wage: " + emp.getWage());  
    }  
}
```

Object Oriented Programming

```
class Employee {  
    private int baseSalary;  
    private int overtime;  
    private int rate;  
  
    // Constructor  
    public Employee(int baseSalary, int overtime, int rate) {  
        this.baseSalary = baseSalary;  
        this.overtime = overtime;  
        this.rate = rate;  
    }  
  
    // Method to calculate overtime pay separately  
    private int calculateOvertimePay() {  
        return overtime * rate;  
    }  
  
    // Method to calculate total wage  
    public int getWage() {  
        return baseSalary + calculateOvertimePay();  
    }  
  
    // Main method  
    public static void main(String[] args) {  
        Employee emp = new Employee(30000, 10, 20);  
        System.out.println("Total Wage: " + emp.getWage());  
    }  
}
```


Object Oriented Programming

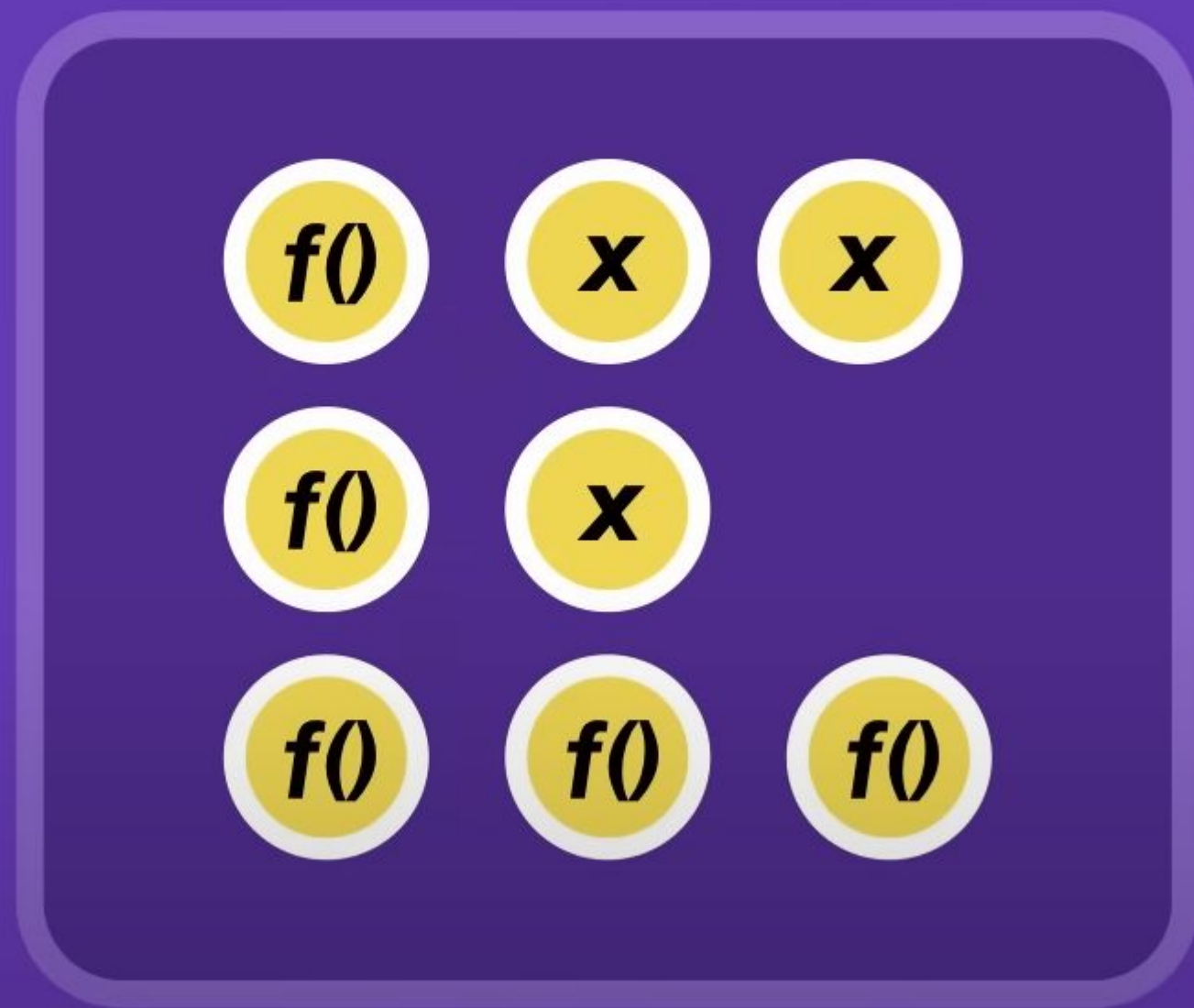
```
class Employee {  
    private int baseSalary;  
    private int overtime;  
    private int rate;  
  
    // Constructor  
    public Employee(int baseSalary, int overtime, int rate) {  
        this.baseSalary = baseSalary;  
        this.overtime = overtime;  
        this.rate = rate;  
    }  
  
    // Method to calculate overtime pay separately  
    private int calculateOvertimePay() {  
        return overtime * rate;  
    }  
  
    // Method to calculate total wage  
    public int getWage() {  
        return baseSalary + calculateOvertimePay();  
    }  
  
    // Main method  
    public static void main(String[] args) {  
        Employee emp = new Employee(30000, 10, 20);  
        System.out.println("Total Wage: " + emp.getWage());  
    }  
}
```

Object Oriented Programming

```
class Employee {  
    private int baseSalary;  
    private int overtime;  
    private int rate;  
  
    // Constructor  
    public Employee(int baseSalary, int overtime, int rate) {  
        this.baseSalary = baseSalary;  
        this.overtime = overtime;  
        this.rate = rate;  
    }  
  
    // Method to calculate overtime pay separately  
    private int calculateOvertimePay() {  
        return overtime * rate;  
    }  
  
    // Method to calculate total wage  
    public int getWage() {  
        return baseSalary + calculateOvertimePay();  
    }  
  
    // Main method  
    public static void main(String[] args) {  
        Employee emp = new Employee(30000, 10, 20);  
        System.out.println("Total Wage: " + emp.getWage());  
    }  
}
```




ABSTRACTION



ABSTRACTION



ABSTRACTION



Simpler Interface

ABSTRACTION

f()

f()

f()

f()

f()

f()

f()

f()

Simpler Interface

Reduce the Impact of Change

TextBox

Select

CheckBox

`hidden`
`innerHTML`

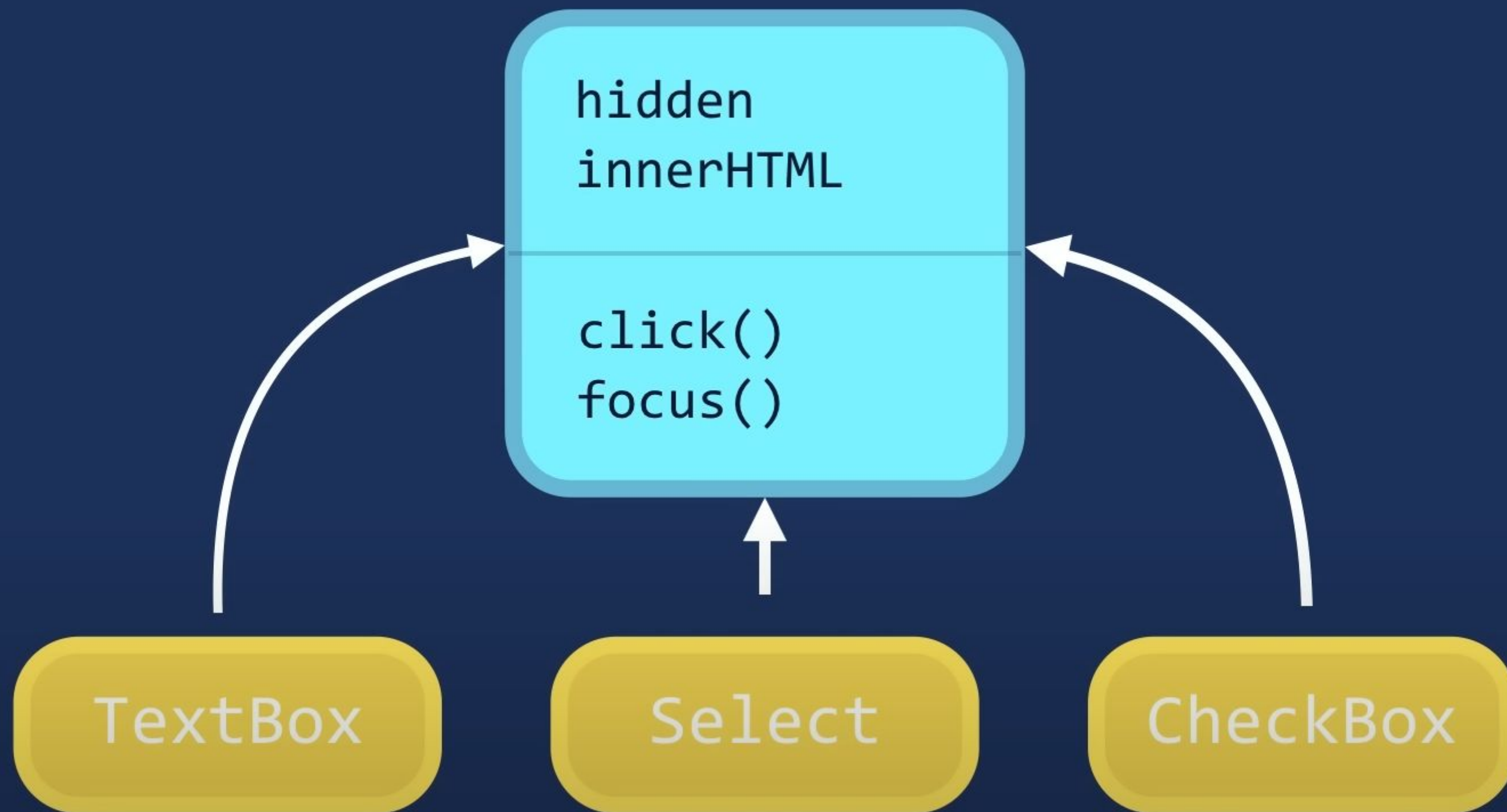
`click()`
`focus()`

TextBox

Select

CheckBox

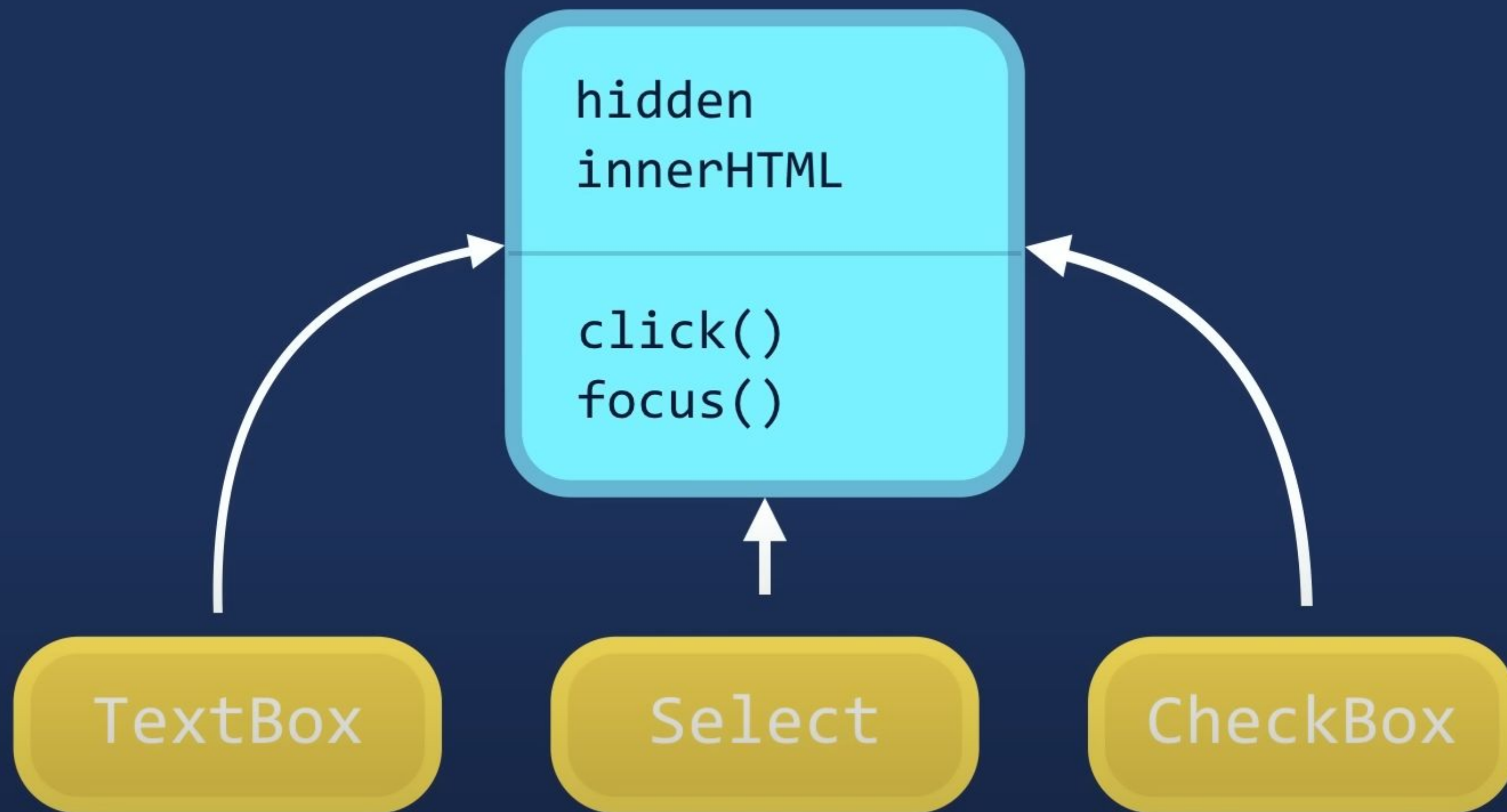
HTMLElement



Polymorphism

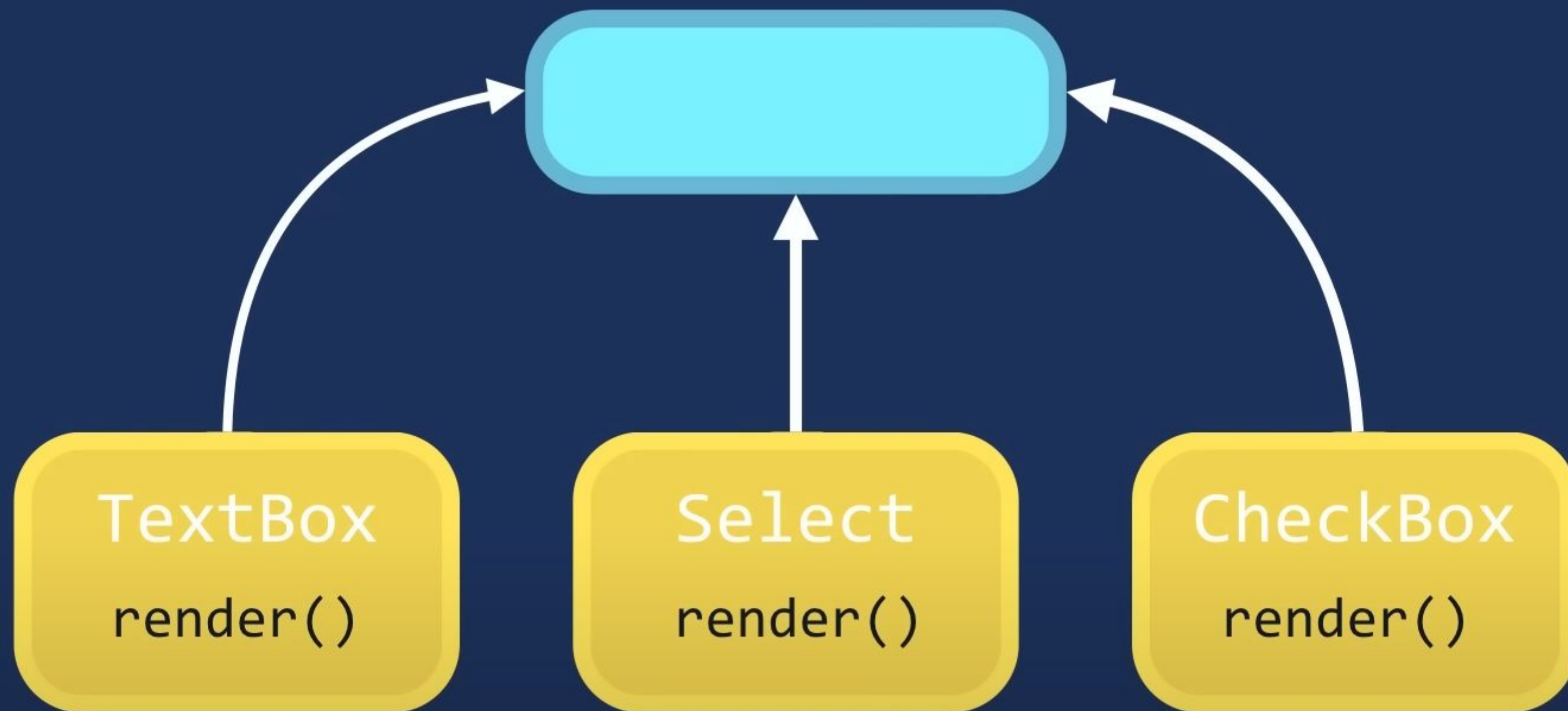
Poly morphism
Many Form

HTMLElement



```
switch (...) {  
    case 'select': renderSelect();  
    case 'text': renderTextBox();  
    case 'checkbox': renderCheckBox();  
  
    case ...  
  
    case ...  
  
    case ...  
  
}
```


HTMLElement



```
switch (...) {  
    case 'select': renderSelect();  
    case 'text': renderTextBox();  
    case 'checkbox': renderCheckBox();  
  
    case ...  
  
    case ...  
  
    case ...  
  
}
```

```
element.render();
```

BENEFITS OF OOP

BENEFITS OF OOP

Encapsulation

BENEFITS OF OOP

Encapsulation

Reduce complexity

BENEFITS OF OOP

Encapsulation

Reduce complexity + increase reusability

BENEFITS OF OOP

Encapsulation

Reduce complexity + increase reusability

Abstraction

BENEFITS OF OOP

Encapsulation

Reduce complexity + increase reusability

Abstraction

Reduce complexity

BENEFITS OF OOP

Encapsulation

Reduce complexity + increase reusability

Abstraction

Reduce complexity + isolate impact of changes

BENEFITS OF OOP

Encapsulation

Reduce complexity + increase reusability

Abstraction

Reduce complexity + isolate impact of changes

Inheritance

BENEFITS OF OOP

Encapsulation

Reduce complexity + increase reusability

Abstraction

Reduce complexity + isolate impact of changes

Inheritance

Eliminate redundant code

BENEFITS OF OOP

Encapsulation

Reduce complexity + increase reusability

Abstraction

Reduce complexity + isolate impact of changes

Inheritance

Eliminate redundant code

Polymorphism

BENEFITS OF OOP

Encapsulation

Reduce complexity + increase reusability

Abstraction

Reduce complexity + isolate impact of changes

Inheritance

Eliminate redundant code

Polymorphism

Refactor ugly switch/case statements